

Hint-Penalized Reinforcement Learning

Process-Level Credit Assignment from Hint Guidance

Xutao Ma

UC Berkeley
Mentor: Sathish Indurthi

With outcome reward alone, a wrong answer teaches nothing

user

Compute $1 + 2 + 3 + \dots + 100$. Let's think step by step and output the final answer within `\boxed{}`.

π_θ

This is the sum of the first 100 positive integers, so I can use the formula for an arithmetic series [reasoning ...] the sum of the first n integers is $\frac{n(n-1)}{2}$, so with $n = 100$ I get $\frac{100-99}{2} = 4950$.

Therefore the answer is 4950 ×

series set up correctly, one wrong formula
 $\Rightarrow$ reward 0

The whole group is scored identically

τ_1	wrong answer $R_1 = 0$	$\left\{ \begin{array}{l} \bar{R} = 0, \quad \sigma_R = 0 \\ A_i = \frac{R_i - \bar{R}}{\sigma_R} = 0 \\ \text{no gradient at all} \end{array} \right.$
τ_2	wrong answer $R_2 = 0$	
τ_3	wrong answer $R_3 = 0$	
τ_4	wrong answer $R_4 = 0$	

What the flat zero hides

A rollout that was one formula away and a rollout that produced nothing usable receive the **same** reward and the **same** update. Even in a mixed group, a failure gets one flat negative number: nothing points at the step that broke.

And for a K -step derivation the all-wrong group is the common case: $P_0 = \prod_k u_k$ decays geometrically in K .

The problem: outcome reward is too sparse

Exponential bottleneck

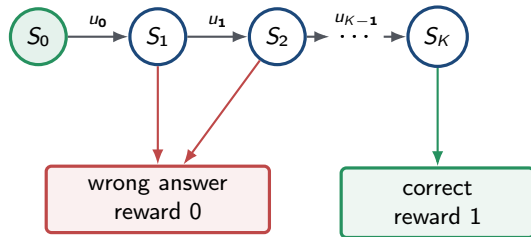
A math solution needs K dependent steps. If step k succeeds with probability u_k , then

$$P_0 = \prod_{k=0}^{K-1} u_k.$$

When $u_k = u < 1$, one correct unaided rollout appears only after about u^{-K} samples.

Final-answer RL cannot distinguish:

- “almost solved but arithmetic error”
- “no useful progress”



Experiment result

Eval Result

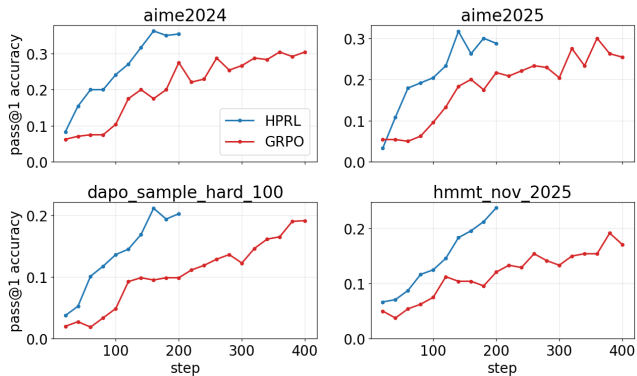


Figure: Eval result comparison of HPRL and GRPO

Substeps define the state

State from ordered substeps

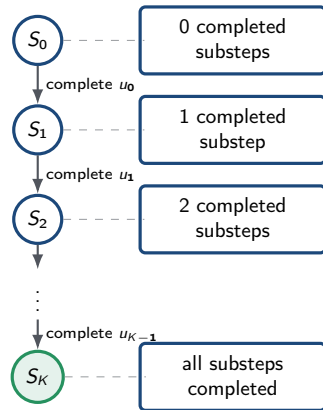
Each problem has ordered canonical substeps

$$\mathcal{U}_q = (u_0, u_1, \dots, u_{K-1}),$$

where u_k is the k -th required **substep**. The progress state is the completed-prefix length:

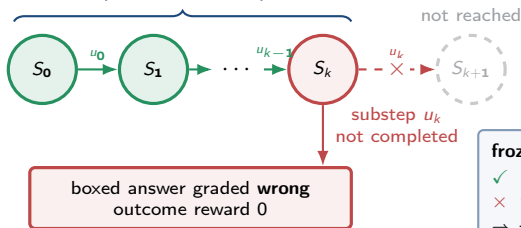
S_k : first k substeps completed, $k = 0, \dots, K$.

S_0 means no verified progress; S_K is solved.



The rollout stalls partway

turn 1: one complete boxed attempt, no assistance

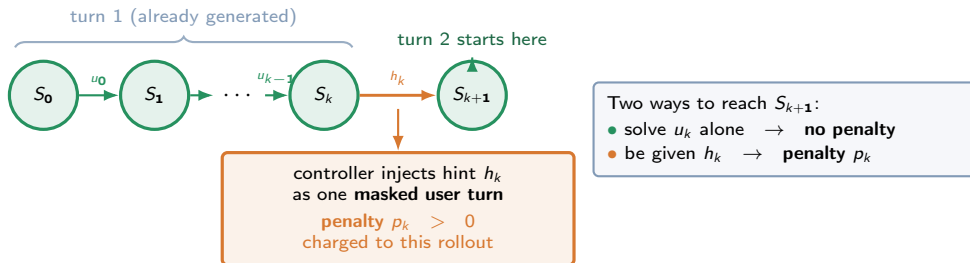


frozen selector f_ϕ reads the whole trace:

- ✓ $u_0, \dots, u_{k-1}$ completed, each with a verbatim quote
- ✗ first unfinished substep: u_k
- ⇒ the rollout's progress state is S_k

The turn reached S_k by itself, then could not cross u_k . Left alone, the rollout stops here: a **wrong final answer**, and k substeps of real progress thrown away.

Inject one hint to cross the blocked substep



The injected message states that the previous answer was wrong, lists the verified progress so far, and supplies h_k . The policy never asks for it and never sees a budget; those tokens carry **zero policy loss**. **Progress is possible, but never free.**

What that looks like in a real rollout

user

Compute $1 + 2 + 3 + \dots + 100$. Let's think step by step and output the final answer within `\boxed{}`.

π_θ

This is the sum of the first 100 positive integers, so I can use the formula for an arithmetic series [reasoning...] the sum of the first n integers is $\frac{n(n-1)}{2}$, so with $n = 100$ I get $\frac{100 \cdot 99}{2} = 4950$.

Therefore the answer is 4950 ×

turn 1 graded wrong; selector names the first unfinished sub-step

controller

Here is a hint to help you make further progress: *Pair the terms from the two ends, $1 + 100, 2 + 99, 3 + 98, \dots$; this gives 50 pairs, each summing to 101.* Using this hint, continue your reasoning and give your final boxed answer.

injected user turn: masked, zero policy loss, penalty p_k

π_θ

I see my error. Pairing the ends gives 50 pairs of 101, so the sum is $50 \times 101 = 5050$ — equivalently $\frac{n(n+1)}{2} = \frac{100 \cdot 101}{2}$, not $\frac{n(n-1)}{2}$.

Hence the answer is 5050 ✓

turn 2 correct: walk reaches S_K , reward $1 - p_k$ instead of 0

One targeted hint turns a dead rollout into a correct walk —
and one flat zero into two turns with different advantages.

Toy problem for legibility; the training corpus is competition-level mathematics.

Where that hint came from: the ordered hint set

h_0 Recognize the target as the arithmetic series $1, 2, \dots, 100$: 100 terms, first 1, last 100.

✓ completed in turn 1
"sum of the first 100 positive integers"

h_1 Pair the terms from the two ends, $1 + 100$, $2 + 99$, $3 + 98, \dots$; this gives 50 pairs, each summing to 101.

⇐ first unachieved
this one is injected

h_2 Multiply the pair count by the pair sum, 50×101 , and report it.

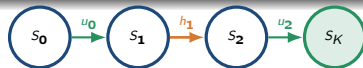
pending
(turn 2 does it unaided)

Difficulty labels give the costs: easy/moderate $\rightarrow w = 1/1.5$, so $p = (0.25, 0.50, 0.25)$ and

$\sum_k p_k = 1$. The hint that mattered is also the expensive one.

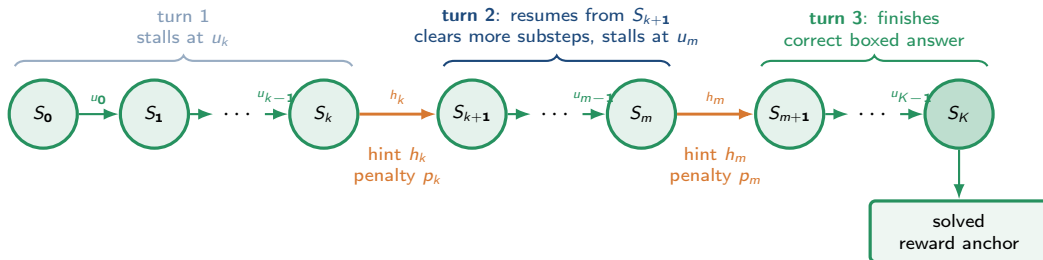
How the frozen selector picks one

- 1 Input: the problem, the **whole** trace so far, and the ordered hints with each marked completed or pending.
- 2 Scan pending hints *in order*; mark the ones the trace already achieved, each backed by a verbatim quote.
- 3 Return the **first hint still not achieved**.



Three hints, so $K = 3$. Completed prefix $\{h_0\}$: the turn stood at S_1 and failed u_1 ; after the injection turn 2 resumes at S_2 .

The rollout continues turn by turn to S_K



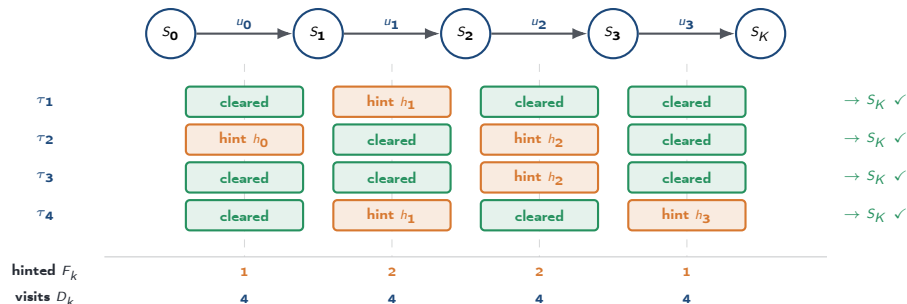
Multi-turn generation, one walk

Each turn resumes from the state the last hint established, so the rollout keeps generating until it either solves the problem or exhausts its hint budget. Here it reaches S_K after **three turns and two hints**.

What we get out of it

The finished rollout is a **complete walk** that visits every state $S_0, \dots, S_K$ and labels each transition as **cleared alone** or **hinted, penalized**. Its reward is the correct-answer reward reduced by the penalties it needed.

N walks per problem, one sample per state each



shown: $N = 4$ walks, $K = 4$ substeps, all four correct ($N = 8$ in the implementation)

Every visit is one transition sample

$$g_{i,k} = \begin{cases} 0, & \text{cleared unaided,} \\ -p_k, & \text{hint } h_k \text{ needed} \end{cases}$$

Counts per state, from the group alone

D_k = walks that reached S_k , F_k = walks that needed h_k , T_k = turn-level failures there. Finished walks visit every state, so $D_k = N$; censored (incorrect) walks only shorten the later columns.

State value estimation from back to forth

Start from the only value we know for sure: a verified correct answer,

$$V_K = 1.$$

To go from V_{k+1} back to V_k

Take the **mean penalty paid on the single transition** $S_k \rightarrow S_{k+1}$ by the N walks:

$$\bar{p}_k = \frac{1}{D_k} \sum_{i: \text{visits } S_k} \text{penalty}_{i,k} = \frac{F_k p_k}{D_k},$$

$$V_k = V_{k+1} - \bar{p}_k$$

Nothing else is needed: one column of the group's outcome grid, N additions, and the value to its right.

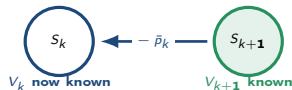
One right-to-left sweep, K subtractions

$$V_0 \leftarrow \cdots \leftarrow V_k \leftarrow V_{k+1} \leftarrow \cdots \leftarrow \underbrace{V_K = 1}_{\text{start here}}$$

the N outcomes at $S_k \rightarrow S_{k+1}$
(one column of the grid)

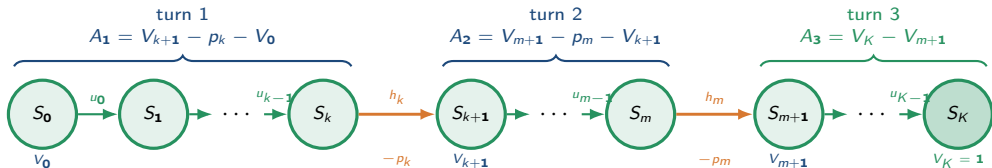
		penalty
τ_1	cleared	0
τ_2	hint h_k	p_k
τ_3	hint h_k	p_k
τ_4	cleared	0

$$\text{mean penalty } \bar{p}_k = \frac{F_k}{D_k} p_k = \frac{2}{4} p_k$$



Since $\bar{p}_k \geq 0$, values only rise toward the goal:
 $V_0 \leq \cdots \leq V_K = 1$. Each V_k is the correct-answer reward
minus the assistance still expected from S_k onward.

Turn advantage estimation



each turn is scored by the value it gained, minus the penalty it triggered.

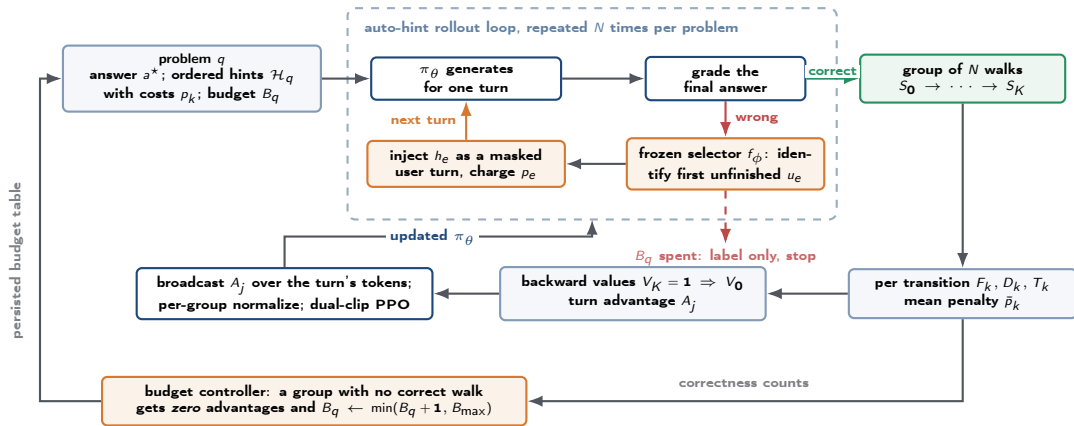
One macro-action per turn

$$A_j^{\text{turn}} = (V_{e_j+q_j} - q_j p_{e_j}) - V_{s_j}.$$

- s_j : state the turn started from; e_j : state verified at its end;
- $q_j = 1$ if a hint was then needed; otherwise $q_j = 0$.

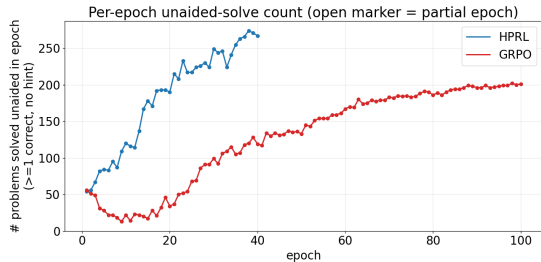
Every trainable token of turn j receives the same A_j ; injected hint tokens receive zero and are masked.

HPRL end to end

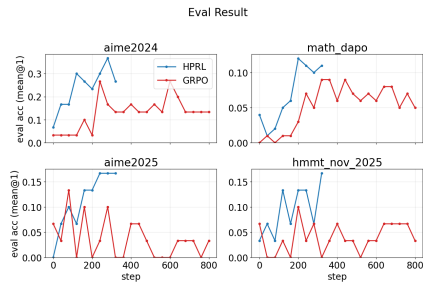


An ordered hint corpus and a frozen selector go in; **process-level advantages** come out. The policy only ever learns to solve — it never sees a hint action, a budget, or a reward model.

Results: HPRL on Qwen3-8B-Base



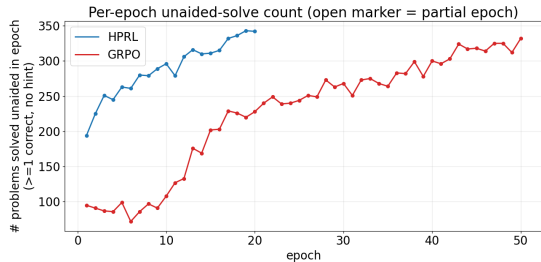
(a) Unaided solved problem per epoch.



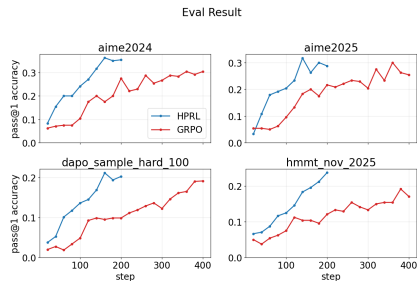
(b) Eval results.

Qwen3-8B-Base training and eval results.

Results: HPRL on Olmo3-7B-Instruct-SFT



(a) Unaided solved problem per epoch.



(b) Eval results.

Olmo3-7B-Instruct-SFT training and eval results.

Thank you for listening

Thank you for listening